LABORATORY OBSERVATION / RECORD

Course:	Full Stack Web Development	**Experiment No.:**	TASK 2
Course Code:	23CM4121	**Date:**	30-08-2026
Student Name:	ROWTHU BHASKAR ABHIRAM	**Roll No.:**	A24126552114

1. TITLE

Dynamic Student Profile Generator Using JavaScript Classes and DOM Manipulation

2. AIM

To create a JavaScript application that defines a Student class, accepts student details entered by the user, and dynamically creates and displays the student profile on the webpage using DOM manipulation when a button is clicked.

3. OBJECTIVE

- To define a JavaScript class Student with properties Name, Roll Number, Department, and CGPA.
- To create an object of the Student class using values entered by the user.
- To select existing DOM elements using document.getElementById().
- To dynamically create new HTML elements using document.createElement().
- To modify the content of elements using textContent.
- To add the dynamically created elements to the webpage using appendChild().
- To handle the button's click event using addEventListener().
- To apply basic CSS styling to the dynamically generated student profile card.

4. THEORY

JavaScript Classes and Objects: A class is a blueprint for creating objects with predefined properties and behaviour. The class keyword defines a constructor that initialises an object's properties when it is created using the new keyword.

DOM (Document Object Model): The DOM represents the HTML page as a tree of objects that JavaScript can read and modify. Methods such as document.getElementById() are used to select existing elements on the page.

Creating Elements Dynamically: document.createElement() creates a new HTML element in memory, which can then be customised (e.g. its text content or CSS class) before being inserted into the page using appendChild().

Modifying Content: The textContent property is used to set or change the text displayed inside an element without interpreting it as HTML, which is safer than using innerHTML for plain text.

Event Handling: addEventListener() attaches a function to an element so that the function runs automatically in response to an event, such as a "click" on a button.

5. ALGORITHM / PROCEDURE

1. Create the HTML page with input fields for Name, Roll Number, Department, and CGPA, a “Display Student Profile” button, and an empty container div with id "profile".
2. Link an external CSS file to style the container, input fields, button, and profile card.
3. In the JavaScript file, define a Student class with a constructor that accepts name, roll, department, and cgpa, and stores them as properties.
4. Select the button using document.getElementById("displayBtn") and attach a click event listener to it.
5. Inside the event handler, read the values entered by the user from each input field using document.getElementById(...).value.
6. Create a new Student object using the values read from the input fields.
7. Clear any previously displayed profile from the "profile" div.
8. Dynamically create a heading and paragraph elements for Name, Roll No, Department, and CGPA using document.createElement(), and set their text using textContent.
9. Append each created element to the "profile" div using appendChild().
10. Apply a CSS class to the profile container so it appears as a styled card.
11. Save the HTML, CSS, and JavaScript files and open the HTML file in a browser to verify the behaviour.
12. Enter sample student details, click the button, and verify the displayed profile against the expected output.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Student Profile</title>
    <!-- Updated CSS link -->
    <link rel="stylesheet" href="css2.css">
</head>
<body>
    <div class="container">
        <h1>Student Profile</h1>
        <input type="text" id="name" placeholder="Enter Name">
        <input type="text" id="roll" placeholder="Enter Roll Number">
        <input type="text" id="department" placeholder="Enter Department">
        <input type="number" id="cgpa" step="0.01" placeholder="Enter CGPA">
        <br><br>
        <button id="displayBtn">Display Student Profile</button>
        <div id="profile"></div>
    </div>
    <!-- Updated JS script link -->
    <script src="script2.js"></script>
</body>
</html>
```

css2.css

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: Arial, sans-serif;
    background: #f2f2f2;
    min-height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
}

.container {
    background: #ffffff;
    width: 400px;
    padding: 35px 40px;
    border-radius: 10px;
    box-shadow: 0 6px 18px rgba(0, 0, 0, 0.15);
    text-align: center;
}

.container h1 {
    margin-bottom: 25px;
    font-size: 28px;
}

.container input {
    width: 100%;
    padding: 12px;
    margin-bottom: 15px;
    border: 1px solid #ccc;
    border-radius: 6px;
    font-size: 14px;
}

.container input:focus {
    outline: none;
    border-color: #0f766e;
}

.container button {
    width: 100%;
    background: #0f766e;
    color: #ffffff;
    border: none;
```

```
    padding: 14px;
    border-radius: 6px;
    font-size: 15px;
    font-weight: bold;
    cursor: pointer;
    transition: 0.3s;
}

.container button:hover {
    background: #0b5a54;
}

#profile {
    margin-top: 20px;
    min-height: 40px;
    background: #e8effc;
    border-radius: 8px;
    text-align: left;
    overflow: hidden;
}

#profile.filled {
    padding: 20px;
}

#profile h2 {
    text-align: center;
    margin-bottom: 15px;
}

#profile p {
    margin: 8px 0;
    font-size: 15px;
    color: #222;
}
```

script2.js

```
// Student class with the required properties
class Student {
    constructor(name, roll, department, cgpa) {
        this.name = name;
        this.roll = roll;
        this.department = department;
        this.cgpa = cgpa;
    }
}

// Select the button using DOM selection
const displayBtn = document.getElementById("displayBtn");

// Event handling: run this function when the button is clicked
displayBtn.addEventListener("click", function () {

    // Read values entered by the user
    const name = document.getElementById("name").value;
    const roll = document.getElementById("roll").value;
    const department = document.getElementById("department").value;
    const cgpa = document.getElementById("cgpa").value;

    // Create an object of the Student class
    const student = new Student(name, roll, department, cgpa);

    // Select the profile container and clear old content
    const profileDiv = document.getElementById("profile");
    profileDiv.innerHTML = "";
    profileDiv.classList.add("filled");

    // Dynamically create the heading element
    const heading = document.createElement("h2");
    heading.textContent = "Student Profile";

    // Dynamically create paragraph elements for each property
    const nameP = document.createElement("p");
    nameP.textContent = "Name : " + student.name;

    const rollP = document.createElement("p");
    rollP.textContent = "Roll No : " + student.roll;

    const deptP = document.createElement("p");
```

```
    deptP.textContent = "Department : " + student.department;

    const cgpaP = document.createElement("p");
    cgpaP.textContent = "CGPA : " + student.cgpa;

    // Add the created elements to the webpage
    profileDiv.appendChild(heading);
    profileDiv.appendChild(nameP);
    profileDiv.appendChild(rollP);
    profileDiv.appendChild(deptP);
    profileDiv.appendChild(cgpaP);
});
```

7. CODE EXPLANATION

Code Element	Description
class Student { constructor(...) {...} }	Defines a blueprint with Name, Roll Number, Department, and CGPA properties, initialised when a new object is created.
new Student(name, roll, department, cgpa)	Creates a Student object using the values entered by the user in the input fields.
document.getElementById("displayBtn")	Selects the button element from the DOM.
addEventListener("click", function(){...})	Registers a click event handler that runs the enclosed code whenever the button is clicked.
document.getElementById("name").value	Reads the current text entered by the user in the corresponding input field.
document.getElementById("profile")	Selects the empty container div where the profile will be displayed.
profileDiv.innerHTML = ""	Clears any previously displayed profile before showing a new one.
document.createElement("h2" / "p")	Dynamically creates new heading and paragraph elements in memory.
element.textContent = "..."	Sets the visible text of a dynamically created element.
profileDiv.appendChild(element)	Inserts the dynamically created element into the profile container, making it visible on the page.
classList.add("filled")	Adds a CSS class that applies padding/spacing to the profile card once it has content.

8. TEST CASES AND EXECUTION DATA

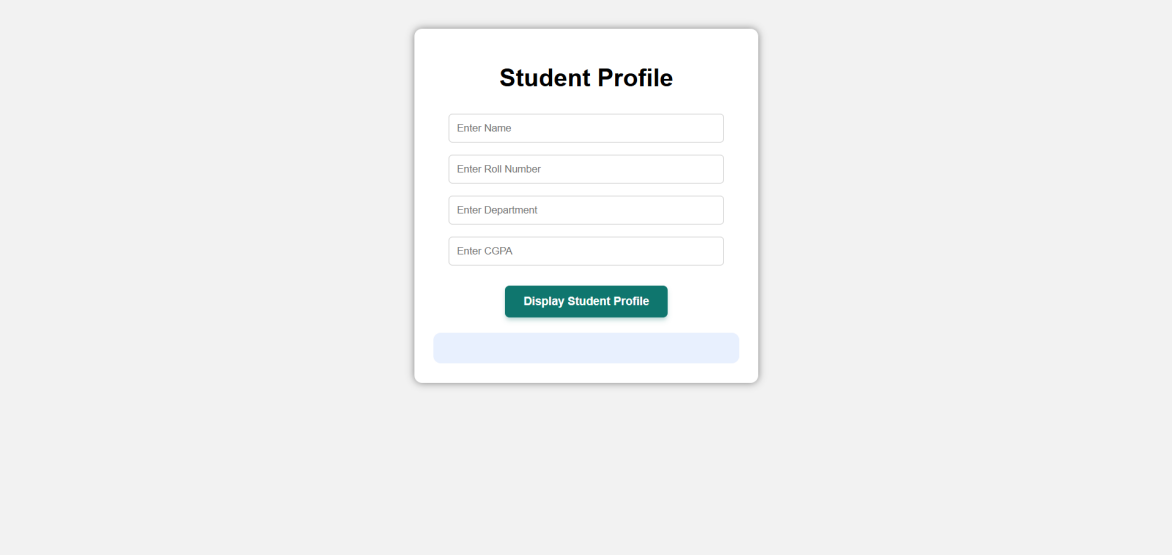
The application was verified in a web browser by entering sample student details and checking the dynamically generated profile against the expected output.

Test Case	Input	Expected Result	Actual Result	Status
TC-01	Name: ROWTHU BHASKAR ABHIRAM, Roll: A24126552114, Dept: CSM, CGPA: 8.6	Profile card displays all four fields correctly	Profile card displayed all four fields correctly	Pass
TC-02	Click Display button with fields empty	Profile card displays with empty values	Profile card displayed with blank values, no error	Pass
TC-03	Click Display button multiple times with different data	Old profile is replaced by the new one, not duplicated	Old profile content replaced correctly	Pass
TC-04	CGPA entered with decimal (e.g. 8.6)	Decimal value displayed exactly as entered	Decimal value displayed correctly	Pass
TC-05	Page load before clicking the button	Profile container remains empty/collapsed	Empty container shown with no profile card	Pass

9. OUTPUT

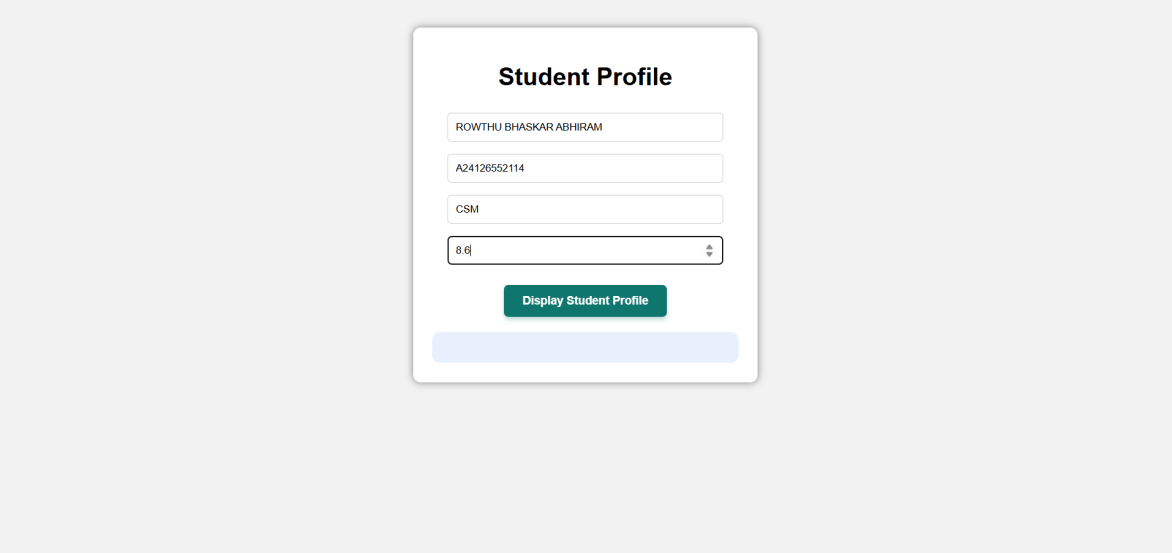
Output 1: Empty Student Profile Form

Displays the initial form with empty input fields for Name, Roll Number, Department, and CGPA, the Display Student Profile button, and the empty profile container below it.



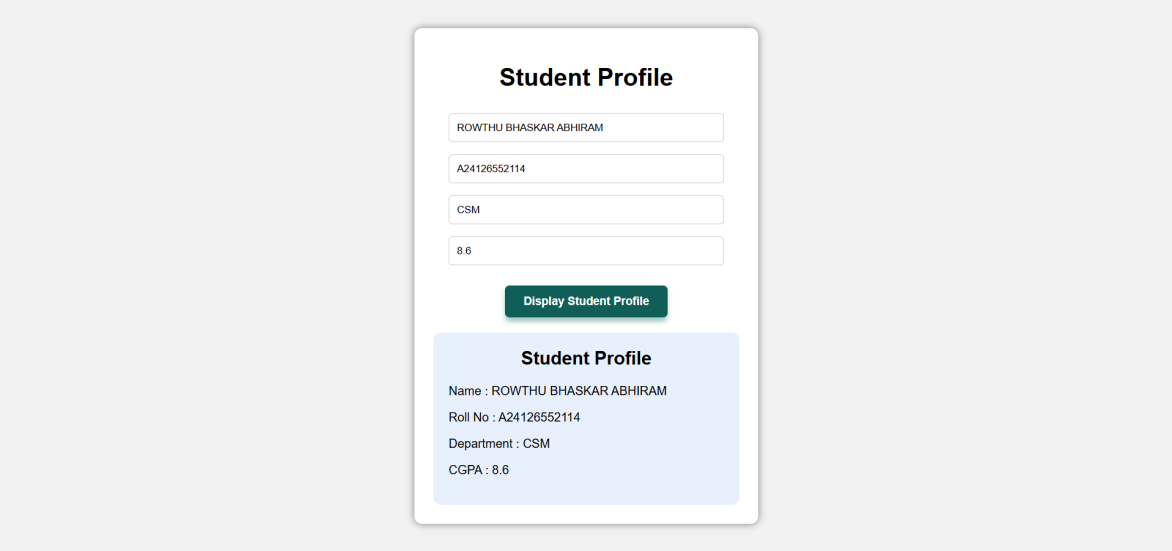
Output 2: Form Filled with Student Details

Displays the form after the user has entered the student's Name, Roll Number, Department, and CGPA, before the Display Student Profile button is clicked.



Output 3: Dynamically Generated Student Profile

Displays the student profile card, created dynamically using DOM manipulation after the Display Student Profile button is clicked, showing the Name, Roll No, Department, and CGPA.



10. RESULT

Thus, a JavaScript application was successfully developed that defines a **Student class** with Name, Roll Number, Department, and CGPA properties, creates a Student object from user-entered values, and dynamically creates and displays the student profile on the webpage using DOM manipulation when the Display Student Profile button is clicked. The profile was styled using CSS and verified to work correctly in a web browser.